

codeHive

Linear Search

THE SEQUENTIAL QUEST

Part of the "Search & Retrieval" Series

codeHive

1. Simple Search Logic

Linear Search is the most straightforward searching algorithm. It works by checking every single element in an array, one by one, until the target value is found or the end of the array is reached.

Unlike sorting algorithms, linear search **leaves the array unchanged**. It is "read-only" in nature.

How it Works:

1. Start at the very first element (index 0).
2. Compare the current element with your **Target Value**.
3. If they match, return the current **index**.
4. If not, move to the next element and repeat.
5. If you reach the end without a match, return **-1**.

2. Manual Visualization

Goal: Find the index of **11**

Initial Array: [12, 8, 9, 11, 5]

INDEX 0 Is 12 == 11? No.



INDEX 1 & 2 Checking 8 and 9... No.

INDEX 3

Is 11 == 11? **Yes!**

12	8	9	11	5
0	1	2	3	4

Result: Search stops. Return index 3.

codeHive

3. Technical Implementation

The code is concise and does not require the array to be sorted beforehand.

```
def linearSearch(arr, targetVal):
    # Iterate through the entire length of the array
    for i in range(len(arr)):
        # Compare current element with target
        if arr[i] == targetVal:
            return i # Match found!

    # If the loop finishes, the value doesn't exist
    return -1

# Usage:
data = [3, 7, 2, 9, 5]
target = 9
result = linearSearch(data, target)

if result != -1:
    print(f"Value found at index {result}")
else:
    print("Value not found")
```

4. Performance Metrics

TIME COMPLEXITY

$O(n)$

Linear Time Growth

Best Case: $O(1)$

The target value is the very first element in the array.

Worst Case: $O(n)$

The target value is at the last index, or does not exist at all (requiring N comparisons).

5. Comparison & Key Differences

Algorithm Class	Primary Purpose	Modifies Data?
Sorting	Organization	Yes (Modifies Index)
Searching	Retrieval	No (Read-Only)

Expert Recommendation

Use Linear Search for **unsorted** or small arrays. If you find yourself searching through the same large dataset frequently, it is better to sort it once (e.g., using Merge Sort) and then use **Binary Search** for a massive performance boost.